



МОСКОВСКИЙ УНИВЕРСИТЕТ ИМЕНИ С.Ю. ВИТТЕ

Факультет Информационных технологий

Кафедра Кафедра информационных систем

Направление подготовки/Специальность Прикладная информатика

ОТЧЕТ

о прохождении

Производственная практика

/вид практики/

Эксплуатационная практика

/тип практики/

Студента Титовой Алины Александровны

(фамилия, имя, отчество)

Место прохождения практики ЧОУВО «МУ им. С.Ю. Витте»

Период прохождения практики с 20.04.2026 г. по 31.05.2026 г.

г. Москва 2026 г.

Оглавление

.....	1
Введение.....	3
Постановка задачи	5
Подзадачи	5
Анализ предметной области	6
Выбор библиотек	Error! Bookmark not defined.
Реализация	9
Скрипт-пипетка	9
Трекбары	10
Морфологическая отчистка.....	11
Детектирование объекта	12
“Исчезающий хвост”	13
Chroma key	14
Система сортировки	15
Вывод	16
Список литературы.	17

Введение

Данная работа посвящена цветовой сегментации изображений, её применению для отслеживания объектов, создания AR-эффектов и сортировки. Тема является актуальной, потому как распознавание объектов по цвету используется на производстве, в котором требуется автоматическая сортировка изделий, а также в медиаиндустрии, где используется технология хромакей и позволяет заменять фон в фильмах и трансляциях.

Главный прием, который рассматривается - работа HSV вместо RGB. Зачастую RGB используется в цифровой среде (мониторы, телевизоры, телефоны) а также описывает цвет комбинацией трех каналов - красного, зеленого, синего. Но для данной задачи RGB подходит плохо т. к. все три канала являются чувствительными к изменению освещения, в то время как HSV устраняет этот недостаток. HSV - описывает цвет через тон (H), яркость (V) и насыщенность (S) и это уже ближе к восприятию цвета человеком, благодаря чему можно точно предсказывать как будет выглядеть цвет при изменении каких либо параметров и позволит выделять объекты даже если освещение будет неравномерным.

В ходе работы будет использоваться язык Python совместно с библиотекой OpenCV. Данная библиотека имеет множество функций таких как, переход из RGB в HSV, чтение, изменение, фильтрация изображения, а также возможность захвата видео с веб-камеры. Данные возможности активно применяются в задачах, где используется Computer Vision и реализация становится возможной без привлечения дополнительных библиотек. Благодаря тому что Open Source Computer Vision Library была спроектирована

для задач реального времени, она является одной из быстрых библиотек данной области, по этой причине и была использована для данной работы.

Также для работы был создан датасет с 30+ изображениями, который содержит в себе объекты различных форм и цветов (красный, желтый, зеленый, синий). Часть изображений имеет равномерное освещение, остальные содержат различные затенения и изгибы света. В кадре используются как одиночные объекты, так и множество сразу, благодаря чему данный датасет подходит для данной работы.

Размерность - входит в число компонентов или признаков, составляющих вектор, т.е. представляет собой размер или длину вектора. Обозначает количество характеристик, которые описывают объект. Также существует размерность вложения (embedding) - обозначает количество чисел в векторе которым модель описывает объект.

Свертка – это математическая операция позволяющая автоматически извлекать признаки из данных (чаще изображений). С ее помощью можно находить шаблоны – границы, углы, цвета, а затем объединять их.

Computer Vision - область искусственного интеллекта, благодаря которой машины могут анализировать визуальные данные. Алгоритмы обучаются на огромных количествах данных и запоминают признаки.

Слой модели – часть функциональной системы или один этап обработки информации, выполняющий конкретную задачу. Например, обработка информации лица, которая выполняется совместно нейронами.

Постановка задачи

Необходимо построить инструмент для цветовой сегментации с трекбарами, при помощи которых будет подбираться подходящий диапазон. На основе этого нужно сделать отслеживание цветного объекта и отрисовку его траектории, старые точки при этом будут затухать – это “исчезающий хвост”. Также нужно реализовать AR-эффект *chroma key*, при нём область выбранного цвета заменяется на фоновое изображение. Дополнительно, разрабатывается система, находит объекты трёх разных цветов и подсвечивает их рамками своего цвета, а также ведет подсчет найденных объектов для цветов.

Подзадачи

1. Разобраться с HSV. Написать скрипт, который по клику мыши отображает значения пикселя и проверить как освещение влияет на BGR и HSV.
2. Сделать окно с трекбарами и реализовать настраивание в реальном времени верхних и нижних границ по H, S и V. Сразу отображать, какие области попадают в маску.
3. Научиться очищать полученную маску. Убирать шум с помощью *opening* и заполнять пустоты через *closing*.
4. Научиться находить объект на маске. Выделить контур, определить его центр и создать окружность.
5. Сделать отслеживание перемещений. Хранить историю центров и рисовать по ним линию с градиентным затуханием старых точек (исчезающий хвост).
6. Сделать эффект *chroma key*. Область выбранного цвета заменяется на заранее установленное фоновое изображение.

7. Обучить программу одновременно видеть объекты трёх цветов, помечать их разными рамками и считать, сколько объектов каждого цвета находится в кадре.

Анализ предметной области

Computer Vision – область ИИ, целью которой является обучение машин понимать визуальные данные на уровне восприятия человека или хотя бы приблизиться к этому максимально. В наше время данная дисциплина охватывает широкий спектр задач, таких как распознавание, генерация изображений, анализ видеопотока и т. д.

Для решения данной задачи необходимо выбрать цветовые диапазоны, а также методы фильтрации, способы трекинга и другие параметры. Основная сложность поставленной задачи это выделение объектов по цвету при световых изменениях в пространстве.

Почему мы не используем RGB? Потому как в нём каждый канал зависит от освещения в отличие от HSV. Он же практически не зависит от освещения, что позволяет задавать диапазон независимо от того, объект на свету или в тени.

Выбор библиотек

Была выбрана библиотека OpenCV, т.к. она имеет множество функций, которые подходят для работы с изображениями в реальном времени. Также позволяет находить контуры объектов, проводить морфологические операции и имеет множество других методов, подходящих для данного задания. Для работы с изображениями и для хранения использует скаляры, векторы, диапазоны и матрицы. Благодаря чему становятся возможны различные преобразования математического характера и множество других действий.

Написана данная библиотека на c++ но также существует на Python, Ruby, JavaScript и др. Так же соответственно работает на windows, linux, mac и т.д.

Применяется в робототехнике, медицине (конкретно технологиях), видеокамерах (для безопасности) и во множестве других сфер.

Выбор OpenCV также удобен тем, что не требует сильной мощности в оборудовании. Все алгоритмы, которые я использую в работе, запускаются на обычном ноутбуке с веб камерой и без особых задержек.

Также специально для задания был создан датасет содержащий 31+ изображение объектов различных цветов с меняющимся освещением в формате png и jpg. В нем находится множество объектов что позволяет удовлетворять поставленную задачу в нахождении объектов по цветам.

Mask(маска) – позволяет выделять на изображениях контуры объектов даже если объект не единственный. Является бинарным изображением, пиксели которого белым (255) выделяют объект, а черные (0) соответствуют фону.

Преимущество макси в том, что по ней сразу видно то, насколько точно настроен диапазон (HSV). Если маска шумная, значит пороги нужно корректировать.

Морфологические операции – методы работы над изображением позволяющее устранять шум, разделять или выделять границы.

Эрозия (Erosion) позволяет реализовать сужение объектов, т. е. способствует удалению шумовых точек.

Дилатация - это расширение объектов, увеличение белых зон и соединение соседних объектов.

Открытие (Opening) – это эрозия, за которой следует дилатация, удаляет выбросы.

Закрытие (Closing) – Дилатация, за которой следует эрозия. Соединяет объекты, которые находятся близко и заполняет черные дыры внутри объектов.

Детектирование объектов – задача, которая находит объекты и определяет их местоположение в машинном обучении. В работе используется “cv2.minEnclosingCircle” данная функция позволяет найти самый маленький круг, в который можно вписать контур, она как бы “обтягивает” объект по размеру.

В реальных minEnclosingCircle удобнее ограничивающего прямоугольника, потому как, окружность не зависит от ракурса объекта т.е его поворота. Какой бы наклон у объекта ни был, центр и радиус будут определяться одинаково.

HSV не привязан к яркости как RGB, но он всё ещё не идеален. Часто пороги могут смещаться из-за изменений в цвете и приходится их перестраивать заново. Нейросетевые детекторы, которые обучены на различных данных, будут более гибкими в этом плане. В том числе у модели могут быть проблемы с захватом фона в маску. Некоторые алгоритмы (вроде grabcut)

будут справляться с этим лучше.

Реализация

Скрипт-пипетка

Скрипт, который при клике на изображение выводит HSV статистику. Данный скрипт использует составленный датасет и при нажатии на любую клавишу, картинка переключается на следующую. При нажатии мышкой(пипеткой) на зону картинки с цветом мы выводим цвет в формате HSV и RGB. Cv2 используем для работы с изображениями, а os для работы с файлами. Получаем файлы, лежащие в папке, после чего реализуем открытие изображений по очереди. Далее преобразуем изображение в HSV и результат сохраняем в переменную (hsv), реализуем событие на клик мыши принимающее координаты нажатия. Преобразованию из BGR в HSV способствует cv2.cvtColor() а print выводит параметры значений BGR и HSV пикселя.

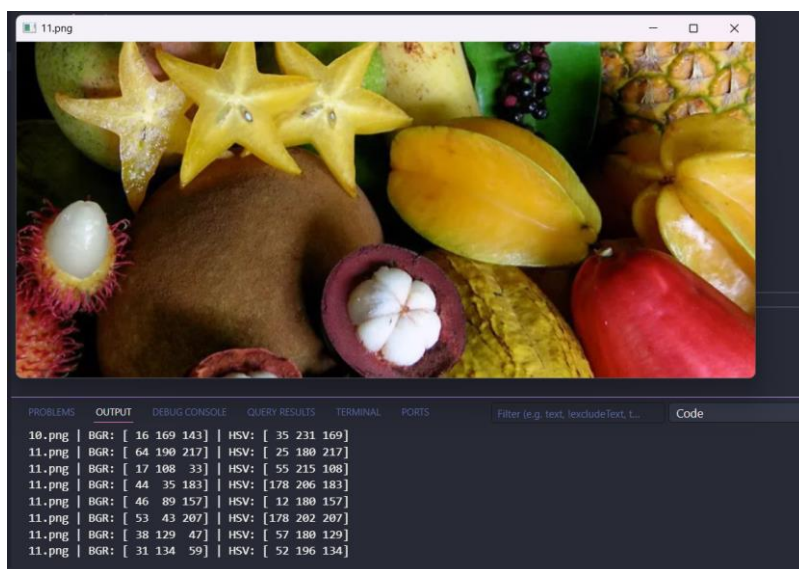


Рис.1

Исходя из работы мы можем сделать вывод что HSV оттенок в отличии от RGB почти не зависит от освещения, а значения BRG сильно меняются. Я пришла к выводу что для данного типа задач где присутствует цветовая фильтрация HSV является наиболее удобным.

Трекбары

Данный скрипт открывает камеру и создает два окна. Первое отображает маску, второе отображает 6 трекбаров (ползунки) и позволяет настраивать диапазоны.

`cv2.VideoCapture(0)` – позволяет подключиться к веб-камере.

Создается окно для трекбаров после чего добавляем ползунок с диапазоном значений (`cv2.createTrackbar()`).

`getTrackbarPos()` позволяет получить текущее значение ползунка. Вновь преобразуем BRG в HSV и при помощи `inRange()` создаем маску по цветовому диапазону.

Так же как можно заметить был создан цикл принимающий новый кадр, и значения всех ползунков. Обновляется он и выводит изображение пока работает программа, это необходимо для получения изменений при движении ползунков.

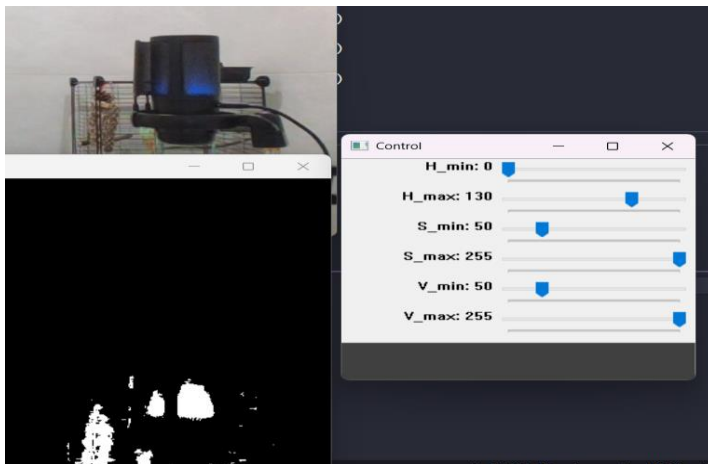


Рис. 2

Данные настройки выделают синий цвет, из-за того, что красный находится на стыке H шкалы, для него необходимо две маски и объединение. Благодаря этому подходу возможно быстро выделять объект по цвету в режиме реального времени.

Морфологическая очистка

Морфологическая очистка маски – к сырой маске применяем opening и closing, отображаем в окнах картинку из датасета, сырую маску, маску после opening и после opening+closing. На сырой маске есть шумы и мелкие дыры, а при помощи opening мы удаляем мелкие шумы, а затем на следующей маске заполняем дыры внутри объектов(closing) но наилучшего результата добиваемся совместным их использованием. В данной реализации важна правильная последовательность потому как если сделать наоборот, шум склеится с объектом и открытием его уже нельзя будет удалить.

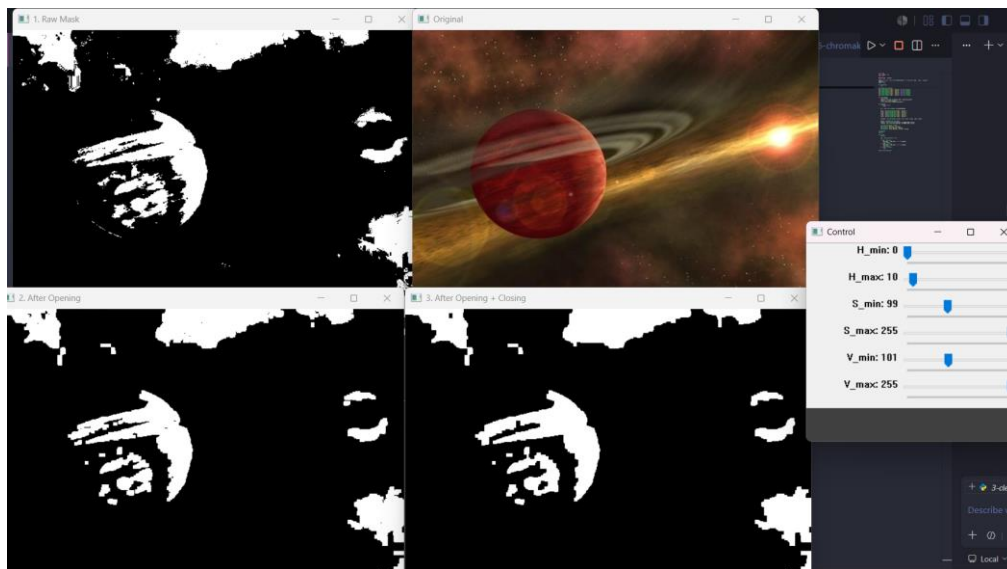


Рис. 3

Переключение между изображениями происходит через s и a.

Также у меня в коде прямоугольное “ядро” матрица 5x5, используется в morphologyEx для морфологических операций. Отвечает за определение того насколько сильно операция затрагивает пиксели вокруг пикселей маски. Используется 5 на 5 т.к. это средний размер, если бы я взяла 7x7 то шума бы было меньше, но края стали бы менее точными и объекты, которые находились бы, рядом могли слипнуться. Так что этот выбор бы являлся ситуативным.

Таким образом мы приходим к выводу что морфологические операции сильно улучшают ситуации в задачах, где требуется очистить маску.

Детектирование объекта

Необходимо найти объект указанного цвета и обвести его окружностью, а также вычислить координаты центра.

Вновь создаем маску, очищаем и реализуем поиск при помощи ползунков `findContours()` находит контур и далее выбирается наиболее большой по площади. Используя `minEnclosingCircle()` строим окружность затем рисуем зеленый круг (`cv2.circle()`). После определения ключевой зоны в терминал выводятся координаты центра. Чтобы спама контурами в терминал сделана проверка на то выбран ли контур в данную секунду.

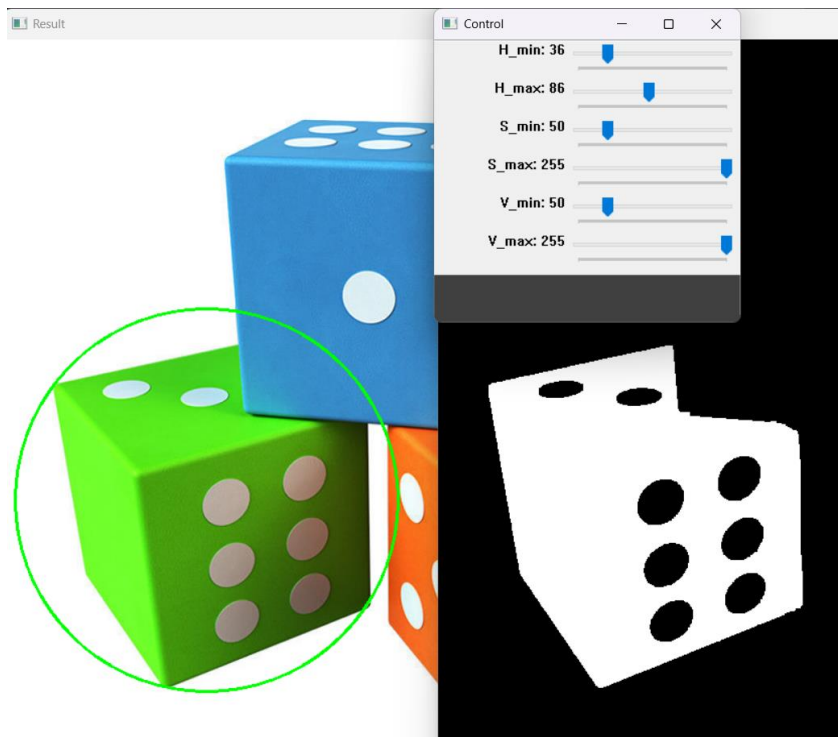


Рис.4 и Рис 5.

Вывод контуров в терминале далее:



Таким образом можно прийти к выводу что детект объекта по цветовому признаку является эффективным методом.

“Исчезающий хвост”

Необходимо реализовать эффект “исчезающего хвоста”. Реализовала это для объекта зеленого цвета и через веб-камеру, по мере движения объекта по траектории движения рисуется красная линия (хвост), имеющая эффект затухания.

Чем старше линия, тем она тусклее по цвету, а координаты объекта мы сохраняем в список(`trajectory`), всего 50 точек.

Новый центр задается в конец списка, `.pop(0)` удаляет самый старый объект, а `line()` рисует линии между точками. Толщина линии от 1 до 4, условие чем меньше значение i (индекса находящегося в списке) тем старше точка и также задается яркость при помощи - $(0, 0, \text{int}(255 * \alpha))$.

Таким образом я реализовала визуализацию, которая позволяет отслеживать траекторию движения объекта зеленого цвета.

Данный эффект “исчезающего хвоста” позволяет наглядно показать, где был объект до этого. Такой подход есть и в отслеживающих движение глаз на экране программах.

Ниже предоставлена реализация работы моей программы.



Рис.6

Chroma key

Этап 6:

Реализация эффекта хромакея, замена зеленого цвета фоном. Часто используется в кино потому как не всё что снимают, можно реализовать бюджетно.

Данный код загружает фоновое изображение, которое предусмотрено, правда стоит учитывать, что размеры изображений разные. Эта проблема была решена тем что фон проходит проверку и автоматически подстраивается под необходимый размер. На деле создается маска зеленого цвета и все зеленые пиксели заменяются на соответствующие им пиксели фона.

`cv2.imread()` загружает фоновое изображение, `resize()` подгоняет под необходимый размер. Замена зеленых пикселей происходит через `frame[mask > 0] = background[mask > 0]`

Ниже до и после(Рис.7 и Рис.8):



Система сортировки

Этап 7:

Система сортировки, создаем маски трех цветов (синий,зеленый,красный). Для красного цвета по опыту используем две маски, затем объединяем их через `cv2.bitwise_or()`, маски у нас тоже очищаются `opening` и `closing`. Для каждого найденного цвета рисуем прямоугольник если объект больше 200 пикселей, соответствующего цвета и размера. Также выводим статистику количества найденных цветов по типу.

`cv2.boundingRect()` – позволяет найти нам координаты прямоугольника вокруг контура а `.rectangle()` рисует сам прямоугольник. Также при помощи `.putText()` выводим текст статистики (кол-во объектов).

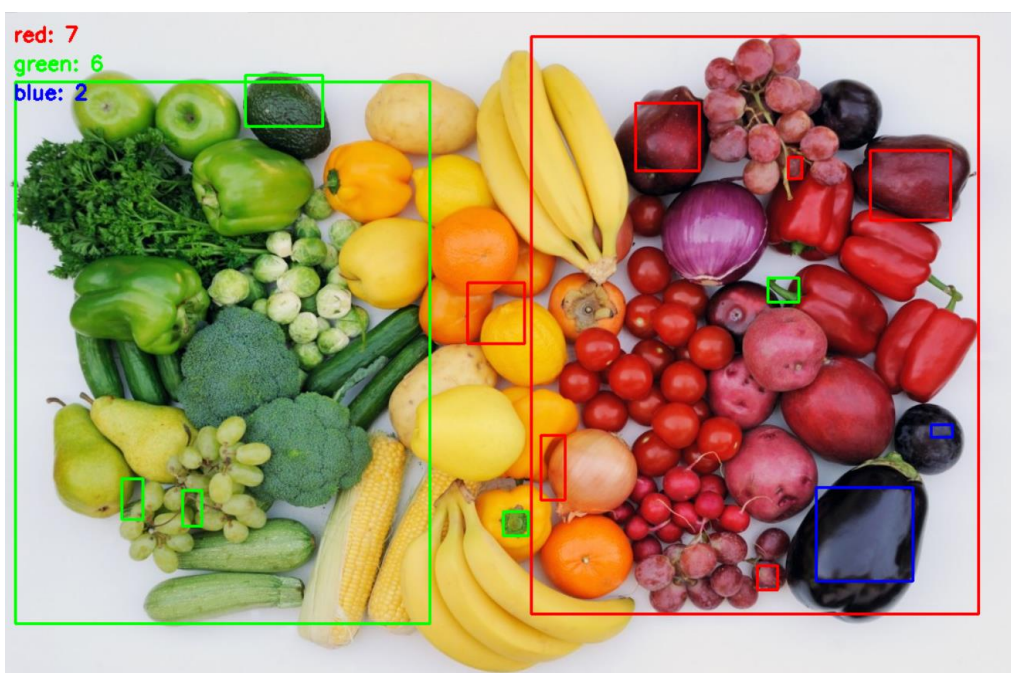


Рис.9

Таким образом я реализовала систему сортировки по цвету, для красного цвета потребовалось вновь объединить две маски. Это простая модель для конвейерного распознавания конкретнее сортировки по цвету.

Используется на производствах, где если условие выполнено и цвет объекта опознан, то срабатывает механизм и т. п.

Вывод

В ходе производственной практики я осваивала Computer Vision технологии и реализовывала различные их методы.

Были выполнены все поставленные задачи такие как: знакомство с HSV, реализация хромакея и сортировщика и д.р.

Я убедилась, что для задач, где требуется цветовая фильтрация удобнее использовать HSV вместо RGB, потому как он лучше улавливает цветовые изменения, также стоило не забывать о добавлении обновления экранной информации. Фактор тени для RGB это реальная проблема, но использование HSV позволило более точно находить объекты по цветам.

Также были проведены различные морфологические операции такие как opening и closing которые позволили удалять шум и удалять дыры в маске. Что является очень удобным инструментом. Также оказалось, что в этом деле важна последовательность, иначе результат будет неудовлетворителен.

Также были реализованы задачи хромакея который зачастую используется в кино и цветовая сортировка, которая нужна на производстве.

В том числе работа шла над трекингом объекта по цвету, это очень интересная функция, которая позволяет увидеть траекторию движения. Полученные в ходе практики навыки могут в дальнейшем использоваться мной в системах видео наблюдения, эффектах и других различных задачах.

Список литературы.

- 1) Способы цветовой сегментации в задачах детектирования дорожных знаков // Хабр URL: <https://habr.com/ru/articles/919088/> (дата обращения: 15.05.2026).
- 2) Морфологические операции // Сообщество Engee URL: <https://engee.com/community/ru/catalogs/projects/morfologicheskie-operatsii-chast-1> (дата обращения: 15.05.2026).
- 3) Компьютерное зрение для начинающих // Хабр URL: <https://habr.com/ru/companies/otus/articles/921402/> (дата обращения: 10.05.2026).
- 4) Детекция объектов в компьютерном зрении (CV) // data light URL: <https://data-light.ru/blog/detekciya-obektov-v-kompyuternom-zrenii/> (дата обращения: 11.05.2026).
- 5) OpenCV // ultralytics URL: <https://www.ultralytics.com/glossary/opencv> (дата обращения: 11.05.2026).
- 6) Что такое цветное пространство: виды и настройка // практикум URL: <https://practicum.yandex.ru/blog/chto-takoe-cvetovoe-prostranstvo-podrobnyj-razbor/> (дата обращения: 14.05.2026).